

Guía de Tecnologías, Arquitectura y Despliegue

Taller de Integración II y Taller de Integración IV

Segundo semestre 2026

1. Objetivo

Esta guía reúne el stack, la arquitectura, la estructura del monorepo y los procedimientos básicos de desarrollo y despliegue de Taller de Integración II (TI2) y Taller de Integración IV (TI4).

La guía sirve para:

- establecer un stack tecnológico común;
- explicar la responsabilidad de cada tecnología;
- justificar las principales decisiones tecnológicas;
- definir la estructura inicial del monorepo;
- explicar cómo ejecutar los proyectos localmente;
- definir cómo se comunican las aplicaciones con el backend;
- establecer cómo se gestionan variables de entorno y secretos;
- explicar cómo desplegar las diferentes partes del sistema;
- establecer claramente las responsabilidades técnicas de TI2 y TI4;
- reducir decisiones técnicas innecesarias durante el desarrollo.

La guía complementa el protocolo de trabajo con Git, GitHub y Linear.

2. Principio general de arquitectura

El producto vive en un único **monorepo GitHub**. El monorepo contiene las aplicaciones Web y Mobile, el backend compartido y los paquetes internos que se incorporen durante el desarrollo.

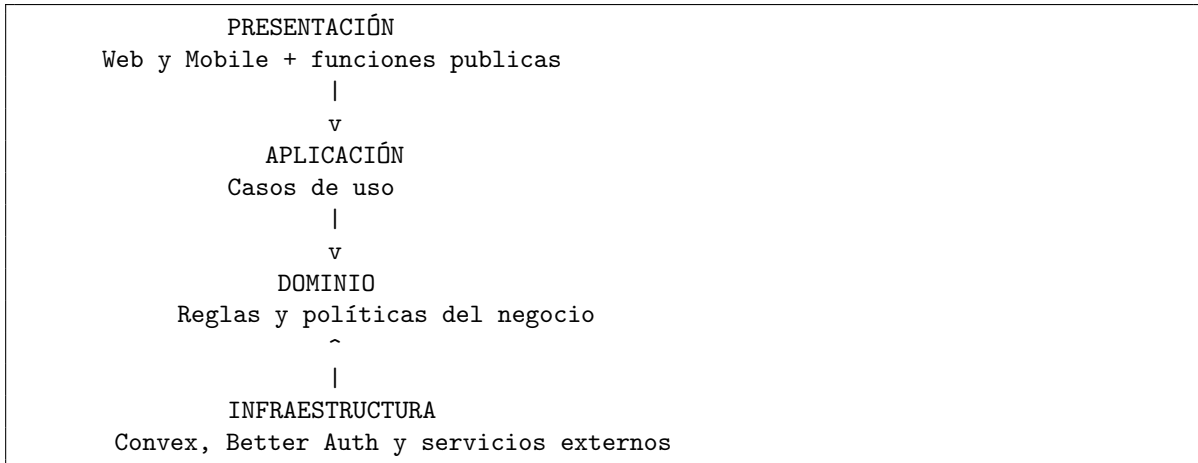
La estructura de carpetas no basta para describir la arquitectura. El backend y ambas aplicaciones cliente siguen una **arquitectura por capas**.

```
Proyecto
|
+-- apps/
| +-- web/ Aplicación Web de TI2
| +-- mobile/ Aplicación Mobile de TI4
|
+-- convex/ Backend, base de datos y autenticación
+-- packages/ Código compartido cuando corresponda
+-- .github/workflows/ Integración continua
+-- package.json Configuración de workspaces
+-- bun.lock Único archivo de bloqueo
```

Web y Mobile comparten:

- el mismo backend;
- la misma base de datos;
- el mismo sistema de autenticación;
- las mismas reglas de negocio.

La arquitectura lógica es:



Una solicitud sigue la dirección **Presentación -> Aplicación -> Dominio**. Infraestructura implementa las interfaces que necesitan Aplicación y Dominio, por lo que puede depender de esas capas. Las reglas del negocio no dependen de Convex, React, Expo ni Better Auth.

2.1. Responsabilidad de cada capa

Presentación: recibe interacciones, valida el formato de entrada, convierte la solicitud en una llamada al caso de uso y presenta el resultado. Incluye pantallas, componentes, routers y funciones públicas de Convex.

Aplicación: implementa casos de uso y coordina el flujo de una operación. Decide qué servicios necesita, controla la autorización de la operación y define los puertos que utilizará para acceder a recursos externos.

Dominio: contiene entidades, objetos de valor, políticas y reglas de negocio. Esta capa debe poder probarse sin levantar React, Expo, Convex ni una base de datos.

Infraestructura: contiene adaptadores concretos para persistencia, autenticación, servicios externos y configuración de ejecución. Aquí se ubican el esquema de Convex, los repositorios, Better Auth y el código generado.

2.2. Reglas de dependencia

Estas reglas rigen las dependencias:

- una capa externa puede utilizar una capa interna, pero no al revés;
- Presentación no accede directamente a la base de datos;
- Aplicación no contiene componentes React ni consultas directas a Convex;
- Dominio no importa librerías de interfaz, infraestructura, variables de entorno ni tipos generados;
- Infraestructura implementa puertos definidos por Aplicación o Dominio;
- Web y Mobile no duplican reglas de negocio que ya pertenecen al backend;

- los paquetes compartidos contienen contratos, tipos o utilidades reutilizables, pero no se convertirán en un depósito de lógica sin dueño.

TI2 es propietario técnico del backend compartido y de sus capas de Aplicación, Dominio e Infraestructura. TI4 consume esos servicios y mantiene la Presentación Mobile desde la misma base de código.

3. Stack tecnológico

El stack es:

Componente	Tecnología
Modelo arquitectónico	Arquitectura por capas
Lenguaje principal	TypeScript
Frontend Web	React
Build Web	Vite
Routing Web	TanStack Router
Aplicación móvil	React Native + Expo
Routing móvil	Expo Router
Backend	Convex
Base de Datos	Convex
Autenticación	Better Auth
Build móvil	EAS Build
Hosting Web	Vercel
Hosting Backend	Convex Cloud
Control de versiones	Git + GitHub
Gestión del trabajo	Linear
Integración continua	GitHub Actions
Organización del código	Monorepo con Bun workspaces

Ejecuta los comandos con **bun** desde la raíz del monorepo, salvo cuando la guía indique explícitamente una aplicación.

Mantén un único gestor de paquetes y un único archivo de bloqueo en la raíz del monorepo. Las aplicaciones y los paquetes internos usan los workspaces definidos en el **package.json** raíz.

3.1. Instalación de Bun

Antes de ejecutar la guía, instala Bun y verifica que esté disponible en la terminal.

En macOS y Linux, ejecuta:

```
curl -fsSL https://bun.sh/install | bash
```

En Windows, ejecuta PowerShell:

```
powershell -c "irm bun.sh/install.ps1|iex"
```

La mayoría de TI2 usa Windows, por lo que esa es la instalación de referencia para el equipo. Después de instalar Bun, abre una nueva terminal y comprueba la instalación con:

```
bun --version
```

En el monorepo no deben coexistir:

```
bun.lock
package-lock.json
pnpm-lock.yaml
yarn.lock
```

4. TypeScript

4.1. Responsabilidad

TypeScript es el lenguaje principal de ambos equipos. Se usa en:

- frontend web;
- aplicación móvil;
- funciones del backend;
- definición del modelo de datos;
- autenticación;
- contratos entre clientes y backend.

4.2. Justificación

Usar TypeScript en todo el proyecto permite compartir tipos entre equipos y detectar muchos errores antes de ejecutar la aplicación.

Además, Convex permite obtener tipos de extremo a extremo entre sus funciones de backend y los clientes TypeScript.

Escribe el código nuevo en TypeScript.

5. React

React construye la interfaz de usuario de la aplicación web de TI2 y pertenece principalmente a la capa de Presentación. React maneja:

- componentes visuales;
- formularios;
- composición de pantallas;
- interacción con el usuario;
- presentación de los datos obtenidos desde los casos de uso;
- captura de eventos y transformación de acciones del usuario en comandos de Aplicación.

React no accede directamente a la base de datos ni contiene reglas de negocio que deban cumplirse de la misma forma en Web y Mobile.

El flujo esperado es:

```
Componente React
|
v
Caso de uso del cliente
|
v
Cliente Convex
|
v
```

```
Función pública de Convex
|
v
Capas de Aplicación y Dominio
```

6. Vite

Vite ejecuta el entorno de desarrollo y construye la aplicación web. Se ocupa de:

- ejecutar el servidor de desarrollo;
- procesar TypeScript y JSX;
- administrar módulos durante desarrollo;
- generar el build de producción.

6.1. Justificación

La aplicación web funciona principalmente como cliente de Convex. Vite se ocupa del desarrollo y la construcción, no de la lógica de negocio. Por eso no se incorpora un segundo framework full-stack con otra capa de servidor.

La distribución del cliente Web queda así:

```
Presentación
|
+-- React
+-- TanStack Router
|
+-- Aplicación del cliente
    |
    +-- Infraestructura del cliente
        |
        +-- Convex Client
        +-- Better Auth Client
```

La lógica de servidor permanecerá en Convex y seguirá la separación entre Presentación, Aplicación, Dominio e Infraestructura definida anteriormente.

7. TanStack Router

TanStack Router administra las rutas de la aplicación web. Se ocupa de:

- definición de páginas;
- navegación;
- parámetros de URL;
- layouts;
- rutas anidadas;
- navegación type-safe.

Las rutas se definen mediante archivos.

Por defecto las rutas se almacenarán en:

```
src/routes/
```

TanStack Router genera automáticamente:

```
src/routeTree.gen.ts
```

No edites este archivo manualmente.

7.1. Justificación

Se usa TanStack Router en lugar de TanStack Start porque Convex ya define el servidor del proyecto. TanStack Start incorporaría funcionalidades de servidor que la arquitectura no necesita.

8. Convex

Convex es la plataforma central del backend. Aporta:

- backend;
- base de datos;
- plataforma de ejecución de funciones;
- mecanismo de sincronización de datos con los clientes.

TI2 desarrolla y mantiene el backend.

Convex no define por sí solo la arquitectura. Sus funciones públicas actúan como adaptadores de Presentación: reciben la solicitud, validan su forma y delegan en un caso de uso. No concentran consultas, autorización y reglas de negocio en una sola función extensa.

8.1. Tipos de funciones

La API pública del backend usa:

Queries: operaciones de lectura.

Mutations: operaciones que modifican datos.

Actions: operaciones que requieren interacción con servicios externos o comportamientos que no corresponden a una query o mutation.

Queries, mutations y actions forman el borde de entrada del backend. El caso de uso que invocan pertenece a Aplicación; las reglas que debe respetar pertenecen al Dominio.

La API se distribuye así:

```
Cliente
|
+-- Query -----> Leer
|
+-- Mutation ----> Modificar
|
+-- Action -----> Servicio externo / proceso especial
```

La lógica de negocio no debe quedar directamente en queries, mutations o actions. Mantén delgadas estas funciones para poder probar y reutilizar el caso de uso sin depender del runtime de Convex.

9. Base de datos Convex

Convex administra el modelo de datos, que pertenece a Infraestructura.

El esquema principal se encontrará en:

```
convex/schema.ts
```

El esquema debe estar bajo control de versiones.

El esquema describe cómo se persisten los datos, pero no reemplaza las entidades ni las reglas del Dominio. Cuando una regla de negocio necesita leer o guardar información, Aplicación usa un puerto e Infraestructura lo implementa con Convex.

Para cada cambio relevante del modelo de datos:

- realizarse mediante una branch;
- pasar por Pull Request;
- considerar el impacto sobre la aplicación web;
- considerar el impacto sobre la aplicación móvil;
- quedar asociados al issue correspondiente en Linear.

No dejes cambios manuales indispensables que solo conozca una persona.

10. Better Auth

Better Auth administra la autenticación de usuarios y pertenece a Infraestructura del backend.

La instancia principal de Better Auth corre sobre Convex.

Sus responsabilidades incluyen:

- registro de usuarios;
- inicio de sesión;
- cierre de sesión;
- manejo de sesiones;
- mecanismos adicionales de autenticación habilitados por el proyecto.

El adaptador de Better Auth entrega la identidad autenticada al borde de entrada. Aplicación y Dominio usan esa identidad para evaluar autorización y políticas del negocio, sin importar directamente módulos de Better Auth.

10.1. Autenticación y autorización

Distingue:

Autenticación: determina quién es el usuario.

Autorización: determina qué puede hacer el usuario.

En consecuencia:

```
Infraestructura
|
+--> Better Auth entrega la identidad
|
v
Presentación
```

```

    |
    v
Aplicación
    |
¿Puede ejecutar esta operación?
    |
    v
Dominio evalúa la política de autorización

```

Implementa las reglas de autorización del negocio en Aplicación o Dominio, según corresponda. No confíes solo en ocultar una pantalla del frontend.

11. Expo y React Native

TI4 desarrolla la aplicación móvil con React Native mediante Expo. React Native pertenece a Presentación Mobile y Expo proporciona el entorno de ejecución.

Expo proporciona:

- entorno de desarrollo móvil;
- herramientas de ejecución;
- configuración de Android e iOS;
- integración con APIs nativas;
- proceso de build mediante EAS.

11.1. Justificación

Expo reduce la configuración nativa que el equipo administra directamente y mantiene la mayor parte del desarrollo en TypeScript y React. Así, ambos equipos comparten tecnologías y conceptos.

TI2 Web	TI4 Mobile
React	React Native
TanStack Router	Expo Router
TypeScript	TypeScript
Convex	Convex
Better Auth	Better Auth

12. Expo Router

Expo Router administra la navegación de la aplicación móvil y pertenece a Presentación.

Las pantallas estarán representadas mediante archivos dentro de:

```
app/
```

Por ejemplo:

```

app/
|
+-- _layout.tsx
+-- index.tsx
+-- login.tsx
|
+-- usuarios/

```



```
|
+-- index.tsx
+-- [id].tsx
```

Expo Router se ocupa de:

- navegación;
- layouts;
- parámetros;
- deep links;
- organización de pantallas.

La lógica de negocio no debe implementarse dentro del router. Las pantallas delegan las operaciones en la Aplicación del cliente o en el cliente de Infraestructura que invoca la API pública.

13. Monorepo y un solo backend

El backend Convex vive únicamente en `convex/`, en la raíz del monorepo. TI4 **no debe crear otra carpeta Convex ni copiar funciones del backend**.

La arquitectura correcta es:

```
Monorepo
|
+-- convex/ Backend
| +-- presentation/ Entrada pública Convex
| +-- application/ Casos de uso
| +-- domain/ Reglas de negocio
| +-- infrastructure/ Adaptadores y persistencia
| +-- schema.ts
|
+-- apps/
    +-- web/ Presentación Web
    +-- mobile/ Presentación Mobile
```

Así existe una sola implementación del servidor y ambas aplicaciones consumen la misma API pública. Cuando el producto crezca, organiza cada capa por funcionalidad si ayuda a mantenerla clara; no mezcles responsabilidades para evitar crear un archivo nuevo.

14. Tipos compartidos dentro del monorepo

Convex genera una representación TypeScript de su API pública. En el monorepo, Web y Mobile acceden a esa representación desde la misma base de código.

El flujo de tipos es:

```
convex/
|
| define funciones y genera tipos
v
convex/_generated/api
| |
v v
apps/web apps/mobile
```

Web y Mobile usan los tipos generados por Convex mediante un alias o un paquete interno definido por el monorepo. La configuración del alias puede variar, pero no dupliques la implementación del backend ni mantengas un contrato generado por separado para cada aplicación.

Estos tipos pertenecen al borde entre Infraestructura y Presentación. No son el modelo de Dominio ni deben hacer que las entidades del negocio dependan de `convex/_generated`. Cuando necesites contratos más estables, usa `packages/shared/` y mantén separadas las reglas del negocio.

Declara la dependencia de Convex en los workspaces correspondientes e instala el paquete desde la raíz:

```
bun add convex --cwd apps/web
bun add convex --cwd apps/mobile
```

Ejecuta el backend desde la raíz:

```
bunx convex dev
```

Mantén `convex/_generated/` según la política del proyecto para archivos generados. Si Convex puede regenerarla automáticamente en cada entorno, no crees una segunda copia manual dentro de `apps/mobile/`.

14.1. Regla

Web y Mobile consumen la API pública definida en `convex/`. TI4 puede consumir las funciones del backend y sus tipos, pero no modifica funciones Convex fuera de su responsabilidad ni duplica en Mobile los casos de uso del backend.

15. Estructura del monorepo

La estructura inicial puede ser:

```
proyecto/
|
+-- apps/
| |
| | +-- web/
| | | +-- src/
| | | +-- presentation/
| | | +-- application/
| | | +-- infrastructure/
| | | +-- public/
| | | +-- package.json
| | | +-- vite.config.ts
| |
| | +-- mobile/
| | +-- app/
| | +-- src/
| | +-- application/
| | +-- infrastructure/
| | +-- package.json
| | +-- app.json
|
+-- packages/
| +-- ui/ Componentes compartidos si corresponde
| +-- shared/ Tipos o utilidades compartidas si corresponde
```

```
|
|-- convex/
| |-- _generated/
| |-- presentation/
| |-- application/
| |-- domain/
| |-- infrastructure/
| |-- betterAuth/
| |-- schema.ts
| |-- auth.config.ts
| |-- convex.config.ts
| |-- http.ts
| |-- ...
|
|-- .github/
| |-- workflows/
|
|-- .env.example
|-- .gitignore
|-- package.json
|-- bun.lock
|-- tsconfig.json
|-- vercel.json
|-- README.md
```

Las carpetas de las capas marcan límites de responsabilidad; no obligan a crear archivos vacíos. Una funcionalidad pequeña puede comenzar con pocos archivos si cada uno tiene una responsabilidad clara y conserva la dirección de dependencias.

Los directorios `packages/ui/` y `packages/shared/` son opcionales. Créelos solo cuando exista una responsabilidad compartida real; no los incorpores como capas vacías por anticipación.

El directorio del backend:

```
convex/_generated/
```

Convex genera este directorio desde su configuración; no lo modifique manualmente.

La aplicación móvil usa el backend y los paquetes compartidos desde este monorepo; no tiene una copia local del backend ni un archivo de contrato externo.

16. Setup inicial del monorepo

Ejecuta todos los comandos siguientes desde la raíz del monorepo.

16.1. 1. Crear la raíz y los workspaces

Declara en la raíz los workspaces de las aplicaciones y de los paquetes internos:

```
{
  "name": "proyecto",
  "private": true,
  "workspaces": [
    "apps/*",
    "packages/*"
  ]
}
```

16.2. 2. Crear aplicación Web

```
bun create vite@latest apps/web -- --template react-ts
bun install
```

16.3. 3. Crear aplicación Mobile

```
bunx create-expo-app@latest apps/mobile
bun install
```

La plantilla estándar de Expo ya incorpora Expo Router.

16.4. 4. Instalar Convex en la raíz

Configura el backend Convex en la raíz del monorepo:

```
bun add convex
bunx convex dev
```

La primera ejecución vincula o crea el proyecto Convex y genera:

```
convex/
```

16.5. 5. Crear los límites de las capas

Después de crear el backend, establece los límites de las capas antes de agregar lógica de negocio:

```
convex/
+-- presentation/ Funciones públicas de Convex
+-- application/ Casos de uso
+-- domain/ Entidades y reglas de negocio
+-- infrastructure/ Convex, autenticación y servicios externos

apps/web/src/
+-- presentation/
+-- application/
+-- infrastructure/

apps/mobile/
+-- app/ Rutas y pantallas
+-- src/application/
+-- src/infrastructure/
```

No crees archivos vacíos en cada carpeta. Decide primero dónde vive la responsabilidad y luego incorpora el código mínimo de la funcionalidad.

16.6. 6. Instalar TanStack Router en Web

```
bun add @tanstack/react-router --cwd apps/web
bun add -d @tanstack/router-plugin --cwd apps/web
bun add -d @tanstack/react-router-devtools --cwd apps/web
```

Agrega en Vite el plugin de TanStack Router antes del plugin de React.

Ejemplo:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import { tanstackRouter } from '@tanstack/router-plugin/vite'

export default defineConfig({
  plugins: [
    tanstackRouter({
      target: 'react',
      autoCodeSplitting: true,
    }),
    react(),
  ],
})
```

16.7. 7. Instalar Better Auth en el backend

```
bun add better-auth @convex-dev/better-auth
```

Realiza la integración dentro del backend Convex y sigue la estructura oficial de Better Auth para Convex.

Entre otros, la integración usa:

```
convex/
|
+-- auth.config.ts
+-- convex.config.ts
+-- http.ts
|
+-- betterAuth/
  |
  +-- convex.config.ts
  +-- auth.ts
  +-- schema.ts
  +-- adapter.ts
```

Genera el esquema de Better Auth con sus herramientas; evita mantenerlo manualmente cuando sea posible.

17. Dependencias de la aplicación Mobile

TI4 usa el cliente Convex y los tipos generados desde la carpeta compartida del backend. No crea ni despliega una segunda implementación.

17.1. 1. Instalar Better Auth para Expo

```
bun add better-auth @better-auth/expo --cwd apps/mobile
bun add expo-network expo-secure-store --cwd apps/mobile
```

Better Auth usa Secure Store para almacenar la sesión de forma apropiada en la aplicación móvil.

Si el proyecto usa proveedores sociales, también puede necesitar dependencias de Expo como linking o web browser.

18. Conexión de la aplicación Web a Convex

La aplicación web usa una instancia de:

```
ConvexReactClient
```

configurada con la URL del deployment.

La variable pública de Vite es:

```
VITE_CONVEX_URL=
```

Dentro del frontend:

```
const convex = new ConvexReactClient(  
  import.meta.env.VITE_CONVEX_URL  
)
```

Provee el cliente en la raíz de la aplicación.

19. Conexión de la aplicación Mobile a Convex

Expo usa el mismo cliente React de Convex.

La variable de Expo es:

```
EXPO_PUBLIC_CONVEX_URL=
```

Configura en la raíz de Expo Router:

```
const convex = new ConvexReactClient(  
  process.env.EXPO_PUBLIC_CONVEX_URL!  
)
```

Luego, provee el cliente a la aplicación mediante el provider correspondiente.

Ambas aplicaciones usan el mismo deployment:

```
Web  
|  
+----\  
    \  
      > mismo deployment Convex  
    /  
Mobile /
```

20. URLs de Convex

Convex usa URLs con responsabilidades distintas.

El proyecto distingue:

Convex URL: conexión del cliente React con las funciones Convex.

Convex Site URL: endpoints HTTP, incluyendo rutas utilizadas por Better Auth cuando corresponda.

Los clientes pueden necesitar variables equivalentes a:

```
VITE_CONVEX_URL=  
VITE_CONVEX_SITE_URL=
```

para Web, y:

```
EXPO_PUBLIC_CONVEX_URL=  
EXPO_PUBLIC_CONVEX_SITE_URL=
```

para mobile.

Estas URLs localizan servicios públicos.

No son el lugar donde deben almacenarse secretos.

21. Variables de entorno

Distingue las variables públicas de los secretos.

21.1. Frontend Web

Vite expone al código cliente variables con prefijo:

```
VITE_
```

Por ejemplo:

```
VITE_CONVEX_URL=
```

Toda variable `VITE_` es visible para el usuario final.

21.2. Aplicación móvil

Expo expone al código cliente las variables con prefijo:

```
EXPO_PUBLIC_
```

Por ejemplo:

```
EXPO_PUBLIC_CONVEX_URL=
```

Toda variable `EXPO_PUBLIC_` es pública.

21.3. Backend

Configura en el entorno de Convex los secretos que usan sus funciones.

Por ejemplo:

```
BETTER_AUTH_SECRET
```

Estos valores no deben estar:

- dentro del código;
- dentro del frontend;
- dentro de variables públicas;
- versionados en Git.

22. Archivos de entorno

No subas al monorepo archivos locales con valores específicos de un desarrollador.

Cada aplicación puede mantener su archivo local dentro de su directorio:

```
apps/web/.env.local
apps/mobile/.env.local
```

Mantén en el monorepo ejemplos de configuración sin secretos:

```
apps/web/.env.example
apps/mobile/.env.example
```

Incluye las variables necesarias, pero ningún secreto.

Ejemplo TI2:

```
VITE_CONVEX_URL=
VITE_CONVEX_SITE_URL=
```

Ejemplo TI4:

```
EXPO_PUBLIC_CONVEX_URL=
EXPO_PUBLIC_CONVEX_SITE_URL=
```

23. Ejecución local del monorepo

Durante el desarrollo ejecuta al menos dos procesos desde la raíz del monorepo.

23.1. Terminal 1: Convex

```
bunx convex dev
```

Este proceso sincroniza los cambios del backend con el deployment de desarrollo.

23.2. Terminal 2: Web

```
bun --cwd apps/web run dev
```

Los procesos se relacionan así:

```
Terminal 1 Terminal 2
bunx convex dev bun --cwd apps/web run dev
  | |
  v v
Backend Convex React + Vite
```

24. Ejecución local de Mobile

Ejecuta la aplicación móvil mediante Expo.

```
bun --cwd apps/mobile run start
```

o equivalentemente:


```
cd apps/mobile && bunx expo start
```

Desde esta sesión puedes ejecutar la aplicación en:

- Android;
- iOS cuando el entorno lo permita;
- dispositivo físico;
- emulador o simulador.

La aplicación Mobile debe conectarse al deployment de desarrollo de Convex definido para el proyecto.

No necesitas ejecutar el backend para modificar exclusivamente Mobile, pero sí debes usar un deployment válido para probar la integración.

25. Ambientes

Distingue al menos:

Development: programación y pruebas durante el desarrollo.

Production: versión estable utilizada para las demostraciones y entregas.

En Mobile también puede existir:

Preview: build destinado a pruebas del equipo.

Usa configuraciones separadas cuando sea necesario.

26. Build de la aplicación Web

Antes de integrar una versión, genera su build de producción.

```
bun --cwd apps/web run build
```

Una aplicación que solo funciona con:

```
bun --cwd apps/web run dev
```

pero cuyo build falla no está lista para integrar.

27. Despliegue del backend Convex

Durante el desarrollo usa:

```
bunx convex dev
```

Para desplegar funciones, esquema e índices en producción usa:

```
bunx convex deploy
```

TI2 es responsable del deployment de producción y lo ejecuta desde la raíz del monorepo.

No todos los integrantes deben manipular producción manualmente.

28. Despliegue de la aplicación Web

Vercel despliega la aplicación web.

Conecta Vercel al único repositorio GitHub y configura `apps/web/` como directorio de la aplicación. Las órdenes de instalación y build deben respetar el `bun.lock` de la raíz.

El flujo general es:

```
GitHub
|
v
main
|
v
Vercel
|
+---- Build apps/web
|
+---- Deploy Convex
|
v
Producción
```

Para integrar el despliegue de Convex con Vercel, configura un deploy key de producción como variable segura:

```
CONVEX_DEPLOY_KEY
```

El build puede ejecutar:

```
bunx convex deploy --cmd 'bun --cwd apps/web run build'
```

Así reduces el riesgo de publicar un frontend incompatible con las funciones Convex correspondientes.

29. Configuración SPA en Vercel

TanStack Router administra las rutas en el cliente.

Por eso, al solicitar directamente una ruta como:

```
/usuarios/123
```

el hosting debe entregar la aplicación y permitir que TanStack Router resuelva la ruta.

El repositorio puede incluir:

```
vercel.json
```

con una regla equivalente a:

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

```
]
}
```

30. Build de la aplicación móvil

EAS Build genera los builds para instalar. Configura EAS con `apps/mobile/` como directorio de la aplicación y comparte el lockfile de la raíz cuando CI se ejecute desde el monorepo.

Primero configura EAS para el proyecto.

Una configuración simplificada de:

```
eas.json
```

puede incluir:

```
{
  "build": {
    "development": {
      "developmentClient": true
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {}
  }
}
```

31. Build Preview de Mobile

Durante el curso no es necesario publicar cada versión en Google Play o App Store.

Para las pruebas usa preferentemente un build interno.

Android:

```
eas build --platform android --profile preview
```

iOS:

```
eas build --platform ios --profile preview
```

EAS genera un build que puedes compartir con integrantes y personas autorizadas para pruebas.

En Android, este mecanismo permite instalar un APK sin publicarlo en Play Store.

32. Build de producción Mobile

Cuando necesites generar una versión de producción:

```
eas build --platform android
```

o:

```
eas build --platform ios
```

También puedes usar:

```
eas build --platform all
```

La publicación efectiva en tiendas es independiente del proceso de desarrollo.

33. Flujos de despliegue

33.1. TI2

```
Monorepo GitHub
|
v
main
|
v
Vercel
|
+--> React/Vite
|
+--> Convex Production
|
v
Web desplegada
```

33.2. TI4

```
Monorepo GitHub
|
v
main
|
v
EAS Build
|
+--> Preview
|
+--> Production
|
v
Aplicación para instalar
```

34. Integración Continua

GitHub Actions verifica los cambios del monorepo antes de integrarlos.

Instala desde la raíz para utilizar el lockfile único:

```
bun install --frozen-lockfile
```

Cuando existan los scripts correspondientes, ejecuta como mínimo estas verificaciones sobre los componentes afectados:

1. instalación de dependencias desde la raíz;
2. chequeo de TypeScript;

3. lint;
4. pruebas unitarias del Dominio y de los casos de uso;
5. pruebas de integración de los adaptadores de Infraestructura cuando corresponda;
6. pruebas de Presentación disponibles;
7. build de Web, Mobile o backend según las rutas modificadas.

La secuencia de CI es:

```

Pull Request
  |
  v
bun install --frozen-lockfile
  |
  v
Type Check
  |
  v
Lint
  |
  v
Tests
  |
  v
Build
  |
  v
Code Review
  |
  v
Merge

```

El despliegue no reemplaza CI.

35. Responsabilidades de TI2

TI2 es responsable técnicamente de:

- desarrollar la aplicación web;
- mantener React y TanStack Router;
- implementar la Presentación pública del backend mediante funciones Convex delgadas;
- implementar los casos de uso de Aplicación;
- definir y mantener las entidades y reglas del Dominio;
- implementar los adaptadores de Infraestructura;
- definir y mantener el modelo de datos;
- implementar Better Auth;
- implementar reglas de autorización;
- desplegar el backend;
- desplegar la aplicación web;
- mantener disponibles los servicios requeridos por TI4;
- mantener estable la API pública consumida por Mobile;
- informar cambios incompatibles;
- mantener actualizados los tipos generados y los paquetes compartidos cuando corresponda.

TI2 es propietario técnico del backend compartido.

36. Responsabilidades de TI4

TI4 es responsable técnicamente de:

- desarrollar la aplicación móvil mediante Expo;
- organizar la navegación mediante Expo Router;
- implementar la Presentación Mobile;
- organizar la Aplicación específica de la interfaz cuando una pantalla requiera más de una llamada al backend;
- integrar la Infraestructura cliente con Convex;
- integrar los flujos de Better Auth;
- consumir los tipos generados desde la configuración compartida;
- detectar y reportar problemas de integración;
- generar builds mediante EAS;
- evitar duplicar lógica perteneciente al backend.

TI4 también mantiene las responsabilidades de gestión, coordinación, seguimiento y control de calidad definidas en el documento metodológico general.

37. Frontera entre TI2 y TI4

La frontera técnica entre ambos equipos está formada por la API pública de Presentación del backend, los casos de uso que expone, el sistema de autenticación compartido y los paquetes internos que consumen ambas aplicaciones.

TI4 no debe:

- crear una segunda base de datos;
- crear una segunda implementación del backend;
- copiar las funciones Convex dentro de `apps/mobile/`;
- modificar directamente producción de TI2;
- duplicar reglas de negocio únicamente para hacer funcionar Mobile.

TI2 debe documentar suficientemente las funciones públicas que consume Mobile.

Como mínimo, documenta:

- propósito;
- argumentos;
- resultado;
- requerimientos de autenticación;
- errores relevantes.

38. Cambios que afectan a ambos equipos

Coordina mediante Linear cualquier cambio que modifique:

- nombre de una función pública;
- argumentos de una función;
- tipo de retorno;
- comportamiento de un caso de uso;
- reglas de autenticación;
- reglas de autorización;

- regla o política del Dominio;
- estructura de datos utilizada por ambos clientes;
- comportamiento funcional esperado por la aplicación Mobile;
- un paquete compartido utilizado por Web y Mobile.

Registra estas dependencias en Linear.

Por ejemplo:

```

TI2-42 Cambiar flujo de autenticación
    |
    +-- bloquea -->
                TI4-18
                Adaptar login mobile

```

39. Qué corresponde a cada capa

Presentación

React, React Native, TanStack Router, Expo Router y las funciones públicas de Convex forman los puntos de entrada de esta capa. Presentación se ocupa de recibir eventos, validar datos de entrada y mostrar estados de carga, éxito y error.

No contiene acceso directo a la base de datos ni reglas de negocio compartidas.

Aplicación

Los casos de uso representan acciones con significado para el producto, por ejemplo *Obtener perfil*, *Registrar usuario* o *Reservar recurso*. Esta capa coordina autenticación, autorización, validaciones de negocio y persistencia mediante puertos.

Un caso de uso no debe importar React, Expo, Convex Client ni componentes de interfaz.

Dominio

El Dominio contiene entidades, objetos de valor, invariantes y políticas del negocio. Es la única capa que debe ser la fuente de verdad para las reglas que deben cumplirse independientemente de Web o Mobile.

Es código TypeScript independiente de frameworks. No recibe directamente documentos de Convex ni tipos de navegación.

Infraestructura

Convex, el esquema, los repositorios, Better Auth, Convex Client y las integraciones con servicios externos forman adaptadores de Infraestructura. Esta capa conoce los detalles técnicos y traduce esos detalles a los puertos que consumen Aplicación y Dominio.

Herramientas de soporte

Vite, Expo, EAS Build, Vercel, Bun y GitHub Actions son herramientas de desarrollo, construcción o despliegue. No son capas de la arquitectura y no deben utilizarse como ubicación para reglas del negocio.

40. Ejemplo de distribución de responsabilidades

Supón que debes implementar:

“Mostrar el perfil de un usuario autenticado.”

La responsabilidad se distribuye así:

```
Presentación
|
+--> Web o Mobile solicita el perfil
|
v
Aplicación
|
+--> caso de uso Obtener perfil
+--> solicita identidad y repositorio
|
v
Dominio
|
+--> valida la política de acceso
|
v
Infraestructura
|
+--> Better Auth identifica al usuario
+--> Repositorio Convex obtiene el perfil
|
+-----+
| |
v v
Web Mobile
React React Native
```

El frontend decide **cómo mostrar** la información. Aplicación y Dominio deciden **qué información puede obtener** el usuario. Infraestructura implementa la forma concreta de obtenerla.

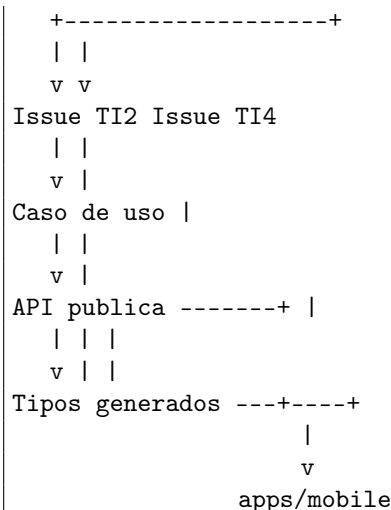
41. Proceso para una funcionalidad compartida

Una funcionalidad que afecte a ambos equipos sigue este flujo:

1. TI4 hace el triage y los equipos identifican las tareas necesarias.
2. TI2 define el caso de uso y las reglas del Dominio.
3. TI2 implementa los puertos y adaptadores de Infraestructura necesarios.
4. TI2 expone una función pública de Convex como entrada del caso de uso.
5. TI2 y TI4 integran el cambio mediante uno o más Pull Requests relacionados.
6. Regeneran o actualizan los tipos públicos de Convex.
7. TI4 implementa el consumo desde la aplicación Mobile.
8. Ambos equipos prueban la integración.
9. Los cambios llegan a los procesos de despliegue de Web y Mobile.

La dependencia entre equipos queda así:

```
Linear
|
```

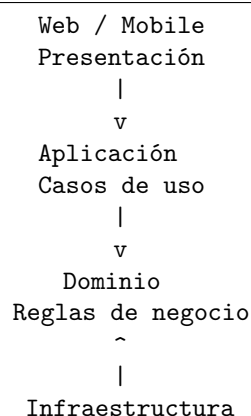
42. Principios técnicos

Durante el desarrollo rigen estos principios:

1. existe un solo backend;
2. existe una sola fuente de verdad para los datos;
3. Web y Mobile consumen los mismos servicios;
4. las dependencias respetan la dirección Presentación → Aplicación → Dominio;
5. el Dominio no depende de frameworks ni proveedores concretos;
6. los casos de uso coordinan las operaciones y las funciones públicas de Convex se mantienen delgadas;
7. la Infraestructura implementa puertos sin trasladar reglas de negocio a los repositorios;
8. los secretos nunca se almacenan en código cliente;
9. las reglas de autorización se validan en el backend;
10. la API pública y los paquetes compartidos deben ser explícitos;
11. los cambios incompatibles deben coordinarse antes de integrarse;
12. el código generado necesario para compilar debe mantenerse correctamente versionado;
13. una aplicación que no puede generar su build no está terminada;
14. los entornos de desarrollo y producción deben mantenerse separados.

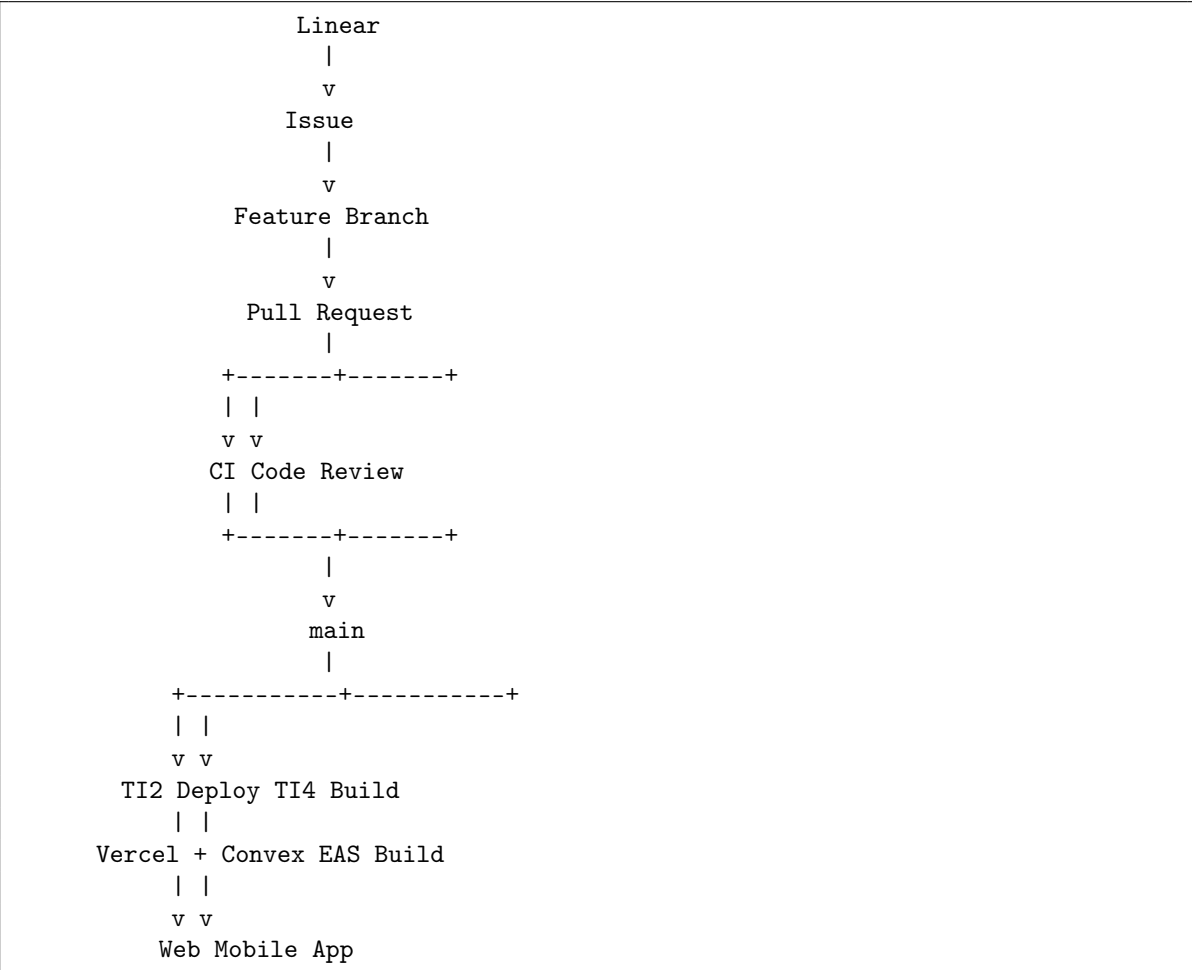
43. Resumen de arquitectura

El stack tecnológico puede resumirse de la siguiente manera:



React, React Native, TanStack Router y Expo Router se ubican principalmente en Presentación. TypeScript aparece en todas las capas. Vite, Expo, EAS Build, Vercel y GitHub Actions soportan el desarrollo y despliegue, pero no contienen reglas de negocio.

El flujo de desarrollo y despliegue es:



44. Referencias oficiales

Las configuraciones específicas pueden cambiar con nuevas versiones de las herramientas. Antes de cambiar una configuración importante, consulta la documentación oficial correspondiente.

- TanStack Router: <https://tanstack.com/router/latest>
- TanStack Router con Vite: [Documentación de TanStack Router con Vite](#)
- Convex: <https://docs.convex.dev/>
- Convex con React: <https://docs.convex.dev/client/react/overview>
- Convex con React Native: <https://docs.convex.dev/quickstart/react-native>
- Convex en Vercel: <https://docs.convex.dev/production/hosting/vercel>
- Better Auth: <https://better-auth.com/>
- Better Auth con Convex: <https://better-auth.com/docs/integrations/convex>
- Better Auth con Expo: <https://better-auth.com/docs/integrations/expo>
- Expo: <https://docs.expo.dev/>
- EAS Build: <https://docs.expo.dev/build/introduction/>

- EAS Internal Distribution: <https://docs.expo.dev/build/internal-distribution/>